

NestUp BD

Complete Code Analysis and Architecture Documentation

Property Rental Platform for Bangladesh

Software Project Lab - 2

Department of Computer Science and Engineering

Islamic University of Technology

September 2, 2025

Contents

1	Executive Summary	5
2	Project Overview	5
2.1	Technology Stack	5
2.2	API Routes Architecture	5
2.2.1	Authentication Routes	5
2.2.2	Main API Routes	6
2.3	Middleware Architecture	8
2.3.1	Authentication Middleware	8
2.3.2	File Upload Middleware	9
3	Frontend Architecture	10
3.1	Application Structure	10
3.1.1	Main Application Component	10
3.2	Context Management	11
3.2.1	Authentication Context	11
3.3	Page Components	11
3.3.1	Home Page	11
3.3.2	Search Page	12
3.3.3	Dashboard System	12
3.3.4	Service Management	13
3.3.5	Payment Processing	13
3.4	Component Architecture	14
3.4.1	Reusable Components	14
3.5	State Management Patterns	15
3.5.1	Local State Management	15
3.5.2	Global State Management	15
4	Security Architecture	15
4.1	Authentication Security	15
4.1.1	JWT Token Management	15
4.1.2	Input Validation and Sanitization	16
4.2	Data Protection	16
4.2.1	Database Security	16
4.2.2	File Upload Security	17
5	Business Logic and Workflows	17
5.1	Service Booking Workflow	17
5.1.1	Booking Creation Process	17
5.1.2	Host Approval Process	18
5.2	Payment Processing Workflow	18
5.2.1	Payment Method Integration	18
5.3	Review and Rating System	18
5.3.1	Review Submission Process	18

6	Configuration and Deployment	19
6.1	Environment Configuration	19
6.1.1	Backend Environment Variables	19
6.1.2	Frontend Configuration	19
6.2	Database Configuration	20
6.2.1	MongoDB Setup	20
6.3	Key Features	20
7	Project Structure	20
7.1	Architecture Principles	21
8	Backend Architecture	21
8.1	Server Entry Point	21
8.1.1	Key Functionalities	21
8.1.2	Server Initialization Flow	22
8.1.3	Middleware Stack	22
8.2	Database Models	22
8.2.1	Model Architecture Overview	22
8.2.2	User Model	23
8.2.3	Service Model	23
8.2.4	Payment Model	25
8.2.5	NestReview Model	25
8.2.6	HostReview Model	26
8.2.7	Booking Model	26
8.2.8	Payment Model	27
8.3	API Routes	28
8.3.1	Authentication Routes	28
8.3.2	Main API Routes	29
8.4	Middleware	30
8.4.1	Authentication Middleware	30
8.4.2	File Upload Middleware	31
9	Frontend Architecture	31
9.1	Application Entry Point	32
9.2	Main App Component	32
9.3	Context Management	32
9.3.1	Authentication Context	32
9.3.2	4.4 Page Components (Client/src/pages/)	33
9.3.3	4.5 Reusable Components (Client/src/components/)	34
9.3.4	4.6 Utility Functions (Client/src/utills/)	35
9.4	5. DATABASE IMPLEMENTATION	35
9.4.1	5.1 MongoDB Connection	35
9.4.2	5.2 Database Schema Design	35
9.4.3	5.3 Data Validation	36
9.5	6. API ENDPOINTS AND DATA FLOW	36
9.5.1	6.1 Authentication Flow	36
9.5.2	6.2 Service Management Flow	36

9.5.3	6.3 Booking and Payment Flow	37
9.5.4	6.4 Wishlist Management	37
9.6	7. AUTHENTICATION SYSTEM	37
9.6.1	7.1 JWT Implementation	37
9.6.2	7.2 Password Security	37
9.6.3	7.3 Route Protection	37
9.7	8. PAYMENT SYSTEM	38
9.7.1	8.1 Payment Methods	38
9.7.2	8.2 Payment Processing	38
9.7.3	8.3 Payment History	38
9.8	9. FILE-BY-FILE ANALYSIS	38
9.8.1	9.1 Backend Files	38
9.8.2	9.2 Frontend Files	39
9.9	10. FRONTEND-BACKEND CONNECTION FLOW	41
9.9.1	10.1 API Communication Pattern	41
9.9.2	10.2 Authentication Integration	41
9.9.3	10.3 Error Handling	41
9.9.4	10.4 Data Transformation	41
9.10	11. CORE FUNCTIONALITIES	41
9.10.1	11.1 User Management	41
9.10.2	11.2 Property Management	41
9.10.3	11.3 Booking System	42
9.10.4	11.4 Payment Processing	42
9.10.5	11.5 Wishlist System	42
9.11	12. MONGODB COLLECTIONS AND SCHEMA	42
9.11.1	12.1 Collection Initialization	42
9.11.2	12.2 Collection Details	42
9.11.3	12.3 Data Relationships	43
9.11.4	12.4 Database Operations	43
9.12	CONCLUSION	43
10	Technical Analysis Summary	43
10.1	Architecture Strengths	44
10.2	Business Logic Implementation	44
10.3	Security and Performance	45
11	Recommendations and Future Enhancements	45
11.1	Technical Improvements	45
11.2	Business Feature Enhancements	46
12	Conclusion	46

1 Executive Summary

This document provides a comprehensive technical analysis of the NestUp BD platform, a modern property rental solution designed specifically for the Bangladeshi market. The platform serves students and professionals seeking accommodation through an intuitive web-based interface that facilitates property listing, searching, booking, and secure payment processing.

2 Project Overview

NestUp BD is a comprehensive property rental platform designed specifically for students and professionals in Bangladesh. The application facilitates property listing, searching, booking, and payment processing with a modern web-based interface.

2.1 Technology Stack

The platform is built using modern web technologies to ensure scalability, maintainability, and optimal performance:

Component	Technology
Frontend Framework	React.js with Vite build tool
Backend Framework	Node.js with Express.js
Database	MongoDB with Mongoose ODM
Authentication	JWT (JSON Web Tokens)
File Upload	Multer middleware
Styling	CSS with custom variables and responsive design
Development Tools	ESLint, Prettier, Nodemon
Version Control	Git with GitHub

Table 1: Technology Stack Overview

2.2 API Routes Architecture

Location: `server/routes/`

The API follows RESTful principles with organized route handlers for different functionalities. Each route file manages specific domain logic with proper error handling and authentication.

2.2.1 Authentication Routes

File: `auth.js`

Purpose: Handles user authentication, registration, and session management.

Method	Endpoint	Purpose	Authentication
POST	/api/auth/register	User registration	None
POST	/api/auth/login	User login	None
POST	/api/auth/logout	User logout	None
GET	/api/auth/me	Get current user	Required
PUT	/api/auth/profile	Update user profile	Required

Table 2: Authentication API Endpoints

Route Endpoints

Registration Flow

1. Validate required fields (name, email, password)
2. Check email format and password length (min 6 characters)
3. Verify email uniqueness in database
4. Hash password using bcrypt
5. Create user record in database
6. Generate JWT token with 7-day expiration
7. Set HTTP-only cookie with security flags
8. Return user data and token

Login Flow

1. Validate email and password presence
2. Find user by email (case-insensitive)
3. Compare provided password with hashed password
4. Generate new JWT token on successful authentication
5. Set secure HTTP-only cookie
6. Return user profile and token

2.2.2 Main API Routes

File: `api.js`

Purpose: Core application functionality including services, bookings, payments, and user management.

Method	Endpoint	Purpose	Authentication
GET	/api/services	List/search services	None
POST	/api/services	Create new service	Required
GET	/api/services/:id	Get service details	None
PUT	/api/services/:id	Update service	Required (Owner)
DELETE	/api/services/:id	Delete service	Required (Owner)
PATCH	/api/services/:id/toggle-active	Toggle service status	Required (Owner)
GET	/api/my-services	Get user's services	Required

Table 3: Service Management API Endpoints

Service Management Endpoints

Service Search and Filtering The GET /api/services endpoint supports comprehensive filtering:

- **Location Filtering:** district and area parameters
- **Price Range:** minPrice and maxPrice parameters
- **Availability Dates:** availableFrom and availableTo parameters
- **Property Type:** propertyType parameter
- **Aggregation Pipeline:** Excludes booked services and includes owner information
- **Population:** Automatic owner details inclusion

Method	Endpoint	Purpose	Authentication
GET	/api/bookings	Get user's bookings	Required
POST	/api/bookings	Create new booking	Required
GET	/api/bookings/:id	Get booking details	Required
PATCH	/api/bookings/:id/status	Update booking status	Required
DELETE	/api/bookings/:id	Cancel booking	Required
GET	/api/bookings/pending-approval	Get pending approvals	Required (Host)
PUT	/api/bookings/:id/approve	Approve booking	Required (Host)
PUT	/api/bookings/:id/reject	Reject booking	Required (Host)

Table 4: Booking Management API Endpoints

Booking Management Endpoints

Booking Creation Validation

- **Self-booking Prevention:** Users cannot book their own services
- **Date Validation:** Start date must be in future, end date after start date
- **Availability Check:** Service must be available for requested period
- **Conflict Detection:** No overlapping bookings for same service

- **Price Calculation:** Automatic total amount computation
- **Status Management:** Proper booking status transitions

Method	Endpoint	Purpose	Authentication
GET	/api/payments	Get user's payments	Required
POST	/api/payments	Create payment	Required
GET	/api/payments/:id	Get payment details	Required
PATCH	/api/payments/:id/status	Update payment status	Required

Table 5: Payment Processing API Endpoints

Payment Processing Endpoints

Payment Method Support

- **Mobile Banking:** bKash, Nagad, Rocket, Upay
- **Traditional Banking:** Bank transfers with account details
- **Credit Cards:** Visa, MasterCard with secure processing
- **Cash Payments:** For in-person transactions
- **Transaction Tracking:** Unique transaction IDs for all methods
- **Status Management:** Complete payment lifecycle tracking

Method	Endpoint	Purpose	Authentication
GET	/api/wishlist	Get user's wishlist	Required
GET	/api/wishlist/check/:serviceId	Check if service in wishlist	Required
POST	/api/wishlist	Add service to wishlist	Required
DELETE	/api/wishlist/:serviceId	Remove from wishlist	Required

Table 6: Wishlist Management API Endpoints

Wishlist Management Endpoints

2.3 Middleware Architecture

Location: *server/middleware/*

2.3.1 Authentication Middleware

File: *auth.js*

Purpose: JWT token validation and user authentication for protected routes.

Key Functions

- **verifyToken:** Validates JWT tokens from cookies or Authorization header
- **generateToken:** Creates new JWT tokens with configurable expiration
- **checkAuth:** Ensures user is authenticated for protected routes

Token Validation Process

1. Extract token from HTTP-only cookie or Authorization header
2. Verify token signature using JWT_SECRET from environment
3. Decode user ID from token payload
4. Fetch user details from database (excluding password)
5. Attach user object to request for route handlers
6. Handle token expiration and invalid token errors

Security Features

- **HTTP-Only Cookies:** Prevents XSS attacks on tokens
- **Secure Flag:** HTTPS-only cookies in production
- **SameSite Policy:** CSRF protection
- **Token Expiration:** 7-day automatic expiration
- **Environment-based Secrets:** Configurable JWT secrets

2.3.2 File Upload Middleware

File: `upload.js`

Purpose: Handles file uploads with validation and storage management.

Configuration

- **Storage Engine:** Disk storage with unique filename generation
- **File Size Limit:** 5MB maximum per file
- **File Type Filtering:** Images only (MIME type validation)
- **Destination:** Uploads directory with automatic creation
- **Filename Strategy:** Timestamp + random suffix for uniqueness

Upload Process

1. Validate file type (images only)
2. Check file size against 5MB limit
3. Generate unique filename with timestamp
4. Store file in uploads directory

- 5. Return file path for database storage
- 6. Handle upload errors gracefully

3 Frontend Architecture

Location: *Client/*
Framework: React 18 with Vite build system
Purpose: Modern, responsive single-page application providing intuitive user interface for property booking and management.

3.1 Application Structure

3.1.1 Main Application Component

File: `App.jsx`
Purpose: Root component managing routing, authentication context, and page transitions.

Key Features

- **React Router:** Client-side routing with nested route support
- **Framer Motion:** Smooth page transitions and animations
- **Authentication Context:** Global user state management
- **Protected Routes:** Conditional rendering based on authentication
- **404 Handling:** Custom not found page for invalid routes

Path	Component	Authentication
/	Home	Public
/search	Search	Public
/dashboard/*	Dashboard	Required
/auth	LoginRegister	Public Only
/provide-service	ProvideService	Required
/host/:id	HostProfile	Public
/payment	Payment	Required
*	NotFound	Public

Table 7: Frontend Route Configuration

Route Configuration

3.2 Context Management

3.2.1 Authentication Context

File: `context/AuthContext.jsx`

Purpose: Centralized authentication state management with persistent user sessions.

State Management

- **User State:** Current user information and authentication status
- **Loading State:** Authentication check progress indicator
- **Persistent Sessions:** Automatic login restoration from cookies
- **Global Access:** `useAuth` hook for component-level access

Key Functions

- **`login()`:** Fetches user data after server-side authentication
- **`logout()`:** Clears user state and calls backend logout
- **`updateUser()`:** Updates local state and syncs with server
- **`checkAuth()`:** Validates current authentication status

Authentication Flow

1. Application loads and checks for existing authentication
2. `AuthContext` calls `/api/auth/me` to validate session
3. User state is populated if valid token exists
4. Components access authentication via `useAuth` hook
5. Login/logout actions update both local and server state
6. Protected routes redirect unauthenticated users

3.3 Page Components

3.3.1 Home Page

File: `pages/Home.jsx`

Purpose: Landing page showcasing platform features and services.

Component Structure

- **Header:** Navigation and user authentication status
- **HeroSection:** Main call-to-action and search functionality
- **ServiceCards:** Featured service categories display
- **HowItWorks:** Platform usage explanation
- **Footer:** Contact information and links

3.3.2 Search Page

File: `pages/Search.jsx`

Purpose: Service discovery with advanced filtering and listing capabilities.

Search Features

- **Location Filtering:** District and area-based search
- **Price Range:** Minimum and maximum price filters
- **Date Availability:** Available from/to date filtering
- **Property Type:** Accommodation type filtering
- **Real-time Results:** Dynamic filtering without page reload
- **Service Cards:** Grid layout with service previews

Data Flow

1. Component mounts and fetches all services from `/api/services`
2. Filter state changes trigger new API requests with query parameters
3. Backend returns filtered results based on search criteria
4. Frontend transforms data and updates listing display
5. `ServiceModal` provides detailed view for selected services

3.3.3 Dashboard System

File: `pages/Dashboard.jsx`

Purpose: Comprehensive user management interface with nested routing.

Route	Purpose
<code>/dashboard</code>	Overview statistics
<code>/dashboard/my-nests</code>	User's listed services
<code>/dashboard/booking-approvals</code>	Host booking management
<code>/dashboard/nest-reviews</code>	Service review management
<code>/dashboard/booked-nests</code>	User's booking history
<code>/dashboard/wishlist</code>	Saved services
<code>/dashboard/payment-history</code>	Transaction records

Table 8: Dashboard Navigation Structure

Dashboard Views

Key Features

- **Responsive Sidebar:** Collapsible navigation with mobile support
- **Nested Routing:** React Router outlet for sub-components
- **Framer Motion:** Smooth transitions between dashboard views
- **Role-based Access:** Different views for hosts and guests
- **Real-time Updates:** Dynamic data refresh for bookings and payments

3.3.4 Service Management

File: `pages/ProvideService.jsx`

Purpose: Multi-step form for creating and listing new services.

Form Steps

1. **Basic Information:** Service title, description, property type
2. **Location Details:** District, area, full address
3. **Pricing:** Daily rate, security deposit, pricing rules
4. **Property Details:** Bedrooms, bathrooms, capacity, size
5. **Amenities:** Available facilities and services
6. **Photos:** Image upload with preview and management
7. **Contact Information:** Host contact details
8. **Terms and Conditions:** Service rules and policies

Validation and Submission

- **Step-by-step Validation:** Real-time form validation
- **Image Upload:** Multiple image support with preview
- **Progress Tracking:** Visual step completion indicator
- **Draft Saving:** Automatic form data persistence
- **Error Handling:** Comprehensive error display and recovery

3.3.5 Payment Processing

File: `pages/Payment.jsx`

Purpose: Secure multi-step payment flow for booking confirmation.

Payment Steps

1. **Personal Information:** Guest details and contact information
2. **Payment Method:** Selection of payment type and details
3. **Confirmation:** Review and final payment submission

Payment Methods

- **Mobile Banking:** bKash, Nagad, Rocket, Upay integration
- **Bank Transfer:** Traditional banking with account details
- **Credit Card:** Secure card processing
- **Cash Payment:** In-person payment option

Security Features

- **Input Validation:** Client-side and server-side validation
- **Secure Transmission:** HTTPS-only payment data
- **Transaction Tracking:** Unique transaction ID generation
- **Error Recovery:** Graceful handling of payment failures

3.4 Component Architecture

3.4.1 Reusable Components

Location: *Client/src/components/*

Core Components

- **Header:** Navigation bar with authentication controls
- **Footer:** Site-wide footer with links and information
- **ServiceCard:** Reusable service display component
- **ServiceModal:** Detailed service view with booking options
- **ListingCard:** Search result display component
- **DashboardSidebar:** Navigation for dashboard views

Form Components

- **LoginRegister:** Authentication form with toggle
- **ReviewForm:** Service review submission
- **BookingForm:** Service booking request form
- **ContactForm:** Host contact information form

Dashboard Components

- **Overview:** Dashboard statistics and summary
- **MyNests:** User's service management interface
- **BookingApprovals:** Host booking approval system
- **BookedNests:** User booking history display
- **Wishlist:** Saved services management

- **PaymentHistory:** Transaction history display

3.5 State Management Patterns

3.5.1 Local State Management

useState Patterns

- **Form Data:** Multi-step form state management
- **Loading States:** API request progress indicators
- **Error States:** Error message display and handling
- **Modal States:** Component visibility and interaction
- **Filter States:** Search and filtering criteria

useEffect Patterns

- **Data Fetching:** API calls on component mount
- **Dependency Updates:** Re-fetching on state changes
- **Cleanup:** Event listener and subscription cleanup
- **Authentication Checks:** User session validation

3.5.2 Global State Management

Context API Usage

- **Authentication Context:** User state and authentication functions
- **Theme Context:** UI theme and styling preferences
- **Notification Context:** Global notification system

State Persistence

- **Local Storage:** User preferences and form drafts
- **Session Storage:** Temporary data and navigation state
- **Cookies:** Authentication tokens and session data

4 Security Architecture

4.1 Authentication Security

4.1.1 JWT Token Management

Token Security Features

- **HTTP-Only Cookies:** Prevents XSS attacks by making tokens inaccessible to JavaScript
- **Secure Flag:** Ensures cookies are only sent over HTTPS in production

- **SameSite Policy:** Provides CSRF protection by restricting cross-site requests
- **Token Expiration:** 7-day automatic expiration reduces long-term exposure
- **Environment Secrets:** JWT secrets stored in environment variables

Password Security

- **bcrypt Hashing:** Industry-standard password hashing with salt
- **Minimum Length:** 6-character minimum password requirement
- **Hash Comparison:** Secure password verification without plaintext storage
- **No Password Exposure:** Passwords excluded from API responses

4.1.2 Input Validation and Sanitization

Server-side Validation

- **Email Format:** Regex validation for email addresses
- **Required Fields:** Mandatory field validation for all forms
- **Data Type Checking:** Type validation for numbers, dates, and strings
- **Length Limits:** Maximum and minimum length constraints
- **File Upload Validation:** MIME type and size restrictions

Client-side Validation

- **Real-time Feedback:** Immediate validation feedback to users
- **Form State Management:** Prevents invalid form submissions
- **Input Sanitization:** XSS prevention through proper escaping
- **File Type Restrictions:** Client-side file type filtering

4.2 Data Protection

4.2.1 Database Security

Connection Security

- **Environment Variables:** Database credentials stored securely
- **Connection Pooling:** Efficient and secure database connections
- **Mongoose ODM:** Built-in query sanitization and validation
- **Schema Validation:** Strict data type enforcement

Data Access Control

- **User Ownership:** Users can only access their own data
- **Service Ownership:** Hosts can only modify their own services
- **Booking Authorization:** Users can only view their own bookings
- **Payment Security:** Secure payment data handling

4.2.2 File Upload Security

Upload Restrictions

- **File Type Filtering:** Only image files allowed (MIME type validation)
- **Size Limits:** 5MB maximum file size per upload
- **Unique Filenames:** Timestamp-based naming prevents conflicts
- **Directory Isolation:** Uploads stored in dedicated directory
- **Path Validation:** Prevents directory traversal attacks

5 Business Logic and Workflows

5.1 Service Booking Workflow

5.1.1 Booking Creation Process

Validation Steps

1. **Authentication Check:** Verify user is logged in
2. **Self-booking Prevention:** Ensure user is not booking own service
3. **Date Validation:** Verify start date is in future and end date is after start
4. **Service Availability:** Check service is active and available
5. **Conflict Detection:** Ensure no overlapping bookings exist
6. **Price Calculation:** Compute total amount based on duration

Status	Description
pending	Initial booking request awaiting host approval
approved	Host has approved the booking request
rejected	Host has rejected the booking request
cancelled	Booking has been cancelled by guest or host
completed	Booking period has ended successfully

Table 9: Booking Status Lifecycle

Booking States

5.1.2 Host Approval Process

Approval Workflow

1. Host receives booking request notification
2. Host reviews booking details and guest information
3. Host can approve or reject with optional message
4. System updates booking status and notifies guest
5. Approved bookings proceed to payment processing
6. Rejected bookings are marked as unavailable

5.2 Payment Processing Workflow

5.2.1 Payment Method Integration

Mobile Banking Integration

- **bKash:** Mobile wallet integration with transaction ID tracking
- **Nagad:** Government-backed mobile financial service
- **Rocket:** Dutch-Bangla Bank mobile banking service
- **Upay:** UCB Fintech mobile payment solution

Payment Validation

1. Validate payment method selection
2. Verify payment amount matches booking total
3. Generate unique transaction ID
4. Process payment through selected method
5. Update payment status based on response
6. Send confirmation to both guest and host

5.3 Review and Rating System

5.3.1 Review Submission Process

Review Types

- **Nest Reviews:** Guests review services after completion
- **Host Reviews:** Hosts review guests after their stay
- **Mutual Reviews:** Both parties can review each other

Review Validation

- **Booking Verification:** Only completed bookings can be reviewed
- **One Review Per Booking:** Prevents multiple reviews for same booking
- **Rating Range:** 1-5 star rating system with validation
- **Content Moderation:** Review content validation and filtering
- **Image Support:** Optional image uploads with reviews

6 Configuration and Deployment

6.1 Environment Configuration

6.1.1 Backend Environment Variables

Variable	Purpose
PORT	Server port configuration (default: 5000)
MONGO_URI	MongoDB connection string
JWT_SECRET	JWT token signing secret
NODE_ENV	Environment mode (development/production)
CORS_ORIGIN	Allowed CORS origins for frontend

Table 10: Backend Environment Variables

Required Environment Variables

6.1.2 Frontend Configuration

Vite Configuration

- **Development Server:** Hot module replacement for fast development
- **Build Optimization:** Production build with code splitting
- **Asset Handling:** Automatic asset optimization and bundling
- **Proxy Configuration:** API proxy for development environment

Package Management

- **npm Scripts:** Standardized build and development commands
- **Dependency Management:** Locked versions for consistent builds
- **Development Dependencies:** Separate dev and production dependencies

6.2 Database Configuration

6.2.1 MongoDB Setup

Connection Configuration

- **Connection Pooling:** Efficient database connection management
- **Retry Logic:** Automatic reconnection on connection failures
- **Timeout Settings:** Configurable connection and query timeouts
- **SSL Support:** Secure connections for production deployments

Index Optimization

- **Email Index:** Unique index on user email for fast lookups
- **Service Indexes:** Location and availability-based indexes
- **Booking Indexes:** User and service-based query optimization
- **Compound Indexes:** Multi-field indexes for complex queries

6.3 Key Features

- **Multi-tenant Architecture:** Support for property owners and tenants
- **Advanced Search:** Location-based filtering with multiple criteria
- **Secure Payments:** Multiple payment methods with transaction tracking
- **Real-time Updates:** Live availability and booking status
- **Mobile Responsive:** Optimized for all device types
- **Admin Dashboard:** Comprehensive management interface

7 Project Structure

The project follows a well-organized modular structure that separates concerns between frontend and backend components:

```
1 NestUp BD/
2     Client/                                # Frontend React application
3         src/
4             components/                    # Reusable UI components
5             pages/                         # Main application pages
6             context/                       # React Context for
state management
7             utils/                         # Utility functions
8             assets/                       # Static assets
9             public/                       # Public static files
10            package.json                   # Frontend dependencies
11            vite.config.js                 # Vite configuration
12            server/                        # Backend Node.js application
```

```
13      models/                                # MongoDB schemas and
      models                                #
14      routes/                               # API route handlers
15      middleware/                           # Custom middleware
      functions
16      uploads/                              # File upload storage
17      server.js                             # Main server entry point
18      package.json                          # Backend dependencies
19      .env                                  # Environment variables
20      Docs/                                 # Documentation files
21      test-payment.js                       # Payment testing script
```

Listing 1: Project Directory Structure

7.1 Architecture Principles

- **Separation of Concerns:** Clear distinction between frontend and backend responsibilities
- **Modular Design:** Components are organized into logical modules for maintainability
- **RESTful API:** Standard HTTP methods and status codes for API communication
- **Security First:** JWT authentication and input validation at all levels
- **Scalable Structure:** Organized codebase that supports future enhancements

8 Backend Architecture

8.1 Server Entry Point

File: `server/server.js`

Purpose: Main application server that initializes the Express app, connects to MongoDB, sets up middleware, and starts the server.

8.1.1 Key Functionalities

- Express server initialization and configuration
- MongoDB connection using Mongoose ODM with connection string from environment
- CORS configuration for cross-origin requests with credentials support
- Cookie parser for JWT token handling in HTTP-only cookies
- JSON body parser with size limits for API requests
- Static file serving for uploaded images and documents
- API route mounting with proper error handling
- Server startup on configurable port (default 3000)
- Environment-based configuration loading
- Graceful error handling and logging

8.1.2 Server Initialization Flow

1. Load environment variables from `.env` file (JWT_SECRET, MONGO_URI, PORT)
2. Create Express application instance with trust proxy settings
3. Configure CORS middleware with credentials and origin validation
4. Set up JSON and URL-encoded body parsers with size limits
5. Initialize cookie parser for secure token handling
6. Establish MongoDB database connection with error handling
7. Mount authentication routes (`/api/auth`)
8. Mount file upload routes (`/api/upload`)
9. Mount review routes (`/api/nest-reviews`, `/api/host-reviews`)
10. Mount main API routes (`/api`)
11. Serve static files from uploads directory
12. Start server and listen on specified port with success logging

8.1.3 Middleware Stack

Middleware	Purpose	Configuration
CORS	Cross-origin requests	credentials: true, origin validation
JSON Parser	Request body parsing	limit: '10mb'
URL Encoded	Form data parsing	extended: true, limit: '10mb'
Cookie Parser	JWT token extraction	HTTP-only cookie support
Static Files	Image/file serving	'/uploads' directory

Table 11: Express Middleware Configuration

8.2 Database Models

Location: `server/models/`

The application uses MongoDB with Mongoose ODM for data modeling. Each model defines comprehensive schemas with validation rules, middleware hooks, and business logic for different entities. The models implement a robust data structure supporting the complete property rental workflow.

8.2.1 Model Architecture Overview

- **User Model:** Authentication, profiles, and user management
- **Service Model:** Property listings with detailed specifications
- **Booking Model:** Reservation management and status tracking
- **Payment Model:** Transaction processing and payment methods

- **NestReview Model:** Property reviews and ratings
- **HostReview Model:** Host/owner reviews and ratings

8.2.2 User Model

File: `User.js`

Purpose: Defines the user schema for authentication and profile management.

Field	Type	Constraints
name	String	Required, trimmed, min 2 chars
email	String	Required, unique, lowercase, email validation
password	String	Required, min 6 chars, hashed
role	Enum	['user', 'admin'], default: 'user'
isVerified	Boolean	Default: true
profile.phone	String	Optional
profile.nidNumber	String	Optional
profile.dateOfBirth	Date	Optional
profile.gender	Enum	['male', 'female', 'other']
profile.occupation	String	Optional
profile.emergencyContact	Object	name and phone
profile.address	Object	street, city, district, postalCode
preferences	Object	User settings
createdAt, updatedAt	Timestamps	Auto-generated

Table 12: User Model Schema

Schema Fields

Methods and Middleware

- `comparePassword()`: Compares plain text password with hashed password
- **Pre-save middleware:** Automatically hashes password before saving

8.2.3 Service Model

File: `Service.js`

Purpose: Defines the property/service schema for rental listings.

Field	Type	Constraints
title	String	Required, max 200 chars
propertyType	Enum	['apartment', 'house', 'room', 'hostel', 'dormitory', 'sublet']
description	String	Required, max 2000 chars
location.district	String	Required
location.area	String	Required
location.address	String	Required
location.coordinates	Object	latitude and longitude
price	Number	Required, min 0
availability.from	Date	Required
availability.to	Date	Required
availability.isAvailable	Boolean	Default: true

Table 13: Service Model - Core Fields

Core Schema Fields

Field	Type	Description
propertyDetails.bedrooms	String	Number of bedrooms
propertyDetails.bathrooms	String	Number of bathrooms
propertyDetails.squareFeet	Number	Property size
propertyDetails.furnishing	Enum	['unfurnished', 'semi-furnished', 'fully-furnished']
amenities	Object	Boolean flags for various amenities
images	Array	Image URLs
thumbnail	String	Main image URL
contact	Object	name, phone, email, whatsapp
owner	ObjectId	Reference to User
rating	Object	average and count
reviews	Array	Review objects
isVerified	Boolean	Default: false
isActive	Boolean	Default: true

Table 14: Service Model - Property Details

Property Details and Features

Methods and Indexes

- **Pre-save middleware:** Updates the updatedAt field automatically
- **Database Indexes:** Created on location fields and price for efficient querying
- **Virtual Properties:** Computed fields for enhanced functionality
- **Validation:** Custom validators for date ranges and price constraints
- **Population:** Automatic owner information inclusion in queries

8.2.4 Payment Model

File: `Payment.js`

Purpose: Manages payment transactions and multiple payment methods for bookings.

Field	Type	Constraints
booking	ObjectId	Reference to Booking (required)
user	ObjectId	Reference to User (required)
amount.base	Number	Required, min 0
amount.discount	Number	Default 0, min 0
currency	String	Default 'BDT'
paymentMethod	Enum	['bkash', 'nagad', 'rocket', 'upay', 'credit_card', 'bank_transfer', 'cash']
status	Enum	['pending', 'processing', 'paid', 'failed', 'cancelled', 'refunded']
transactionId	String	Unique transaction identifier

Table 15: Payment Model - Core Fields

Core Schema Fields

Field	Type	Description
paymentDetails.mobileNumber	String	For mobile banking (bKash, Nagad, etc.)
paymentDetails.bankName	String	For bank transfers
paymentDetails.accountNumber	String	Bank account details
paymentDetails.cardLast4	String	Last 4 digits of credit card
paymentDetails.cardBrand	String	Credit card brand (Visa, MasterCard)
processedAt	Date	Payment processing timestamp
failureReason	String	Error details for failed payments
refundAmount	Number	Amount refunded (if applicable)
refundedAt	Date	Refund processing timestamp

Table 16: Payment Model - Method-Specific Fields

Payment Method Details

8.2.5 NestReview Model

File: `NestReview.js`

Purpose: Manages property reviews and ratings from tenants.

Field	Type	Constraints
service	ObjectId	Reference to Service (required)
user	ObjectId	Reference to User/reviewer (required)
booking	ObjectId	Reference to Booking (required)
rating	Number	Required, min 1, max 5, allows decimals
comment	String	Required, max 1000 chars
images	Array	Optional review images
categories.cleanliness	Number	1-5 rating for cleanliness
categories.location	Number	1-5 rating for location
categories.amenities	Number	1-5 rating for amenities
categories.value	Number	1-5 rating for value for money
categories.accuracy	Number	1-5 rating for listing accuracy

Table 17: NestReview Model Schema

Review Schema Fields

8.2.6 HostReview Model

File: `HostReview.js`

Purpose: Manages host/owner reviews and ratings from tenants.

Field	Type	Constraints
host	ObjectId	Reference to User/host (required)
user	ObjectId	Reference to User/reviewer (required)
booking	ObjectId	Reference to Booking (required)
rating	Number	Required, min 1, max 5, allows decimals
comment	String	Required, max 1000 chars
images	Array	Optional review images
categories.communication	Number	1-5 rating for communication
categories.responsiveness	Number	1-5 rating for responsiveness
categories.helpfulness	Number	1-5 rating for helpfulness
categories.reliability	Number	1-5 rating for reliability
categories.professionalism	Number	1-5 rating for professionalism

Table 18: HostReview Model Schema

Host Review Schema Fields

8.2.7 Booking Model

File: `Booking.js`

Purpose: Manages property booking information and status.

Field	Type	Constraints
service	ObjectId	Reference to Service (required)
user	ObjectId	Reference to User (required)
startDate	Date	Required
endDate	Date	Required
duration	Object	days and months
basePrice	Number	Required
totalAmount	Number	Required
fees	Object	service fee, cleaning fee, security deposit
status	Enum	['pending', 'confirmed', 'active', 'completed', 'cancelled', 'rejected']
paymentStatus	Enum	['pending', 'partial', 'paid', 'refunded', 'failed']

Table 19: Booking Model - Core Fields

Core Schema Fields

Field	Type	Description
guestInfo	Object	numberOfGuests and specialRequests
contactInfo	Object	phone and email (required)
confirmationCode	String	Auto-generated unique identifier
confirmedAt	Date	Timestamp when booking was confirmed
createdAt	Date	Auto-generated creation timestamp
updatedAt	Date	Auto-updated modification timestamp

Table 20: Booking Model - Additional Fields

Additional Information Fields

Methods and Business Logic

- **confirm()**: Updates status to confirmed and sets confirmedAt timestamp
- **checkAvailability()**: Static method to check date conflicts with existing bookings
- **Pre-save middleware**: Automatically calculates duration and generates confirmation code
- **Virtual property**: durationInDays for easy access to booking duration
- **Status management**: Comprehensive booking lifecycle tracking

8.2.8 Payment Model

File: `Payment.js`

Purpose: Handles payment processing and transaction records.

Field	Type	Constraints
booking	ObjectId	Reference to Booking (required)
user	ObjectId	Reference to User (required)
service	ObjectId	Reference to Service (required)
amount	Number	Required, min 0
currency	String	Default: 'BDT'
paymentMethod	Enum	['mobile_banking', 'bank_transfer', 'credit_card', 'cash']
paymentDetails	Object	Method-specific payment details
status	Enum	['pending', 'processing', 'completed', 'failed', 'refunded', 'cancelled']

Table 21: Payment Model - Core Fields

Core Transaction Fields

Field	Type	Description
personalInfo	Object	User's personal information for payment
termsAccepted	Boolean	Required consent flag
processedAt	Date	Timestamp when payment was processed
failureReason	String	Reason for payment failure
gatewayResponse	Object	Payment gateway response data
refund	Object	Refund details and status
createdAt	Date	Auto-generated creation timestamp
updatedAt	Date	Auto-updated modification timestamp

Table 22: Payment Model - Additional Fields

Additional Transaction Data

Payment Processing Methods

- **markAsCompleted()**: Updates status to completed and sets processedAt timestamp
- **markAsFailed()**: Sets status to failed with detailed failure reason
- **processRefund()**: Handles refund processing and status updates
- **getPaymentStats()**: Static method for payment analytics and reporting
- **Gateway Integration**: Seamless integration with multiple payment gateways

8.3 API Routes

Directory: **server/routes/**

Purpose: Defines RESTful API endpoints for client-server communication.

8.3.1 Authentication Routes

File: **auth.js**

Purpose: Handles user authentication, registration, and session management.

Method	Endpoint	Purpose	Protection
POST	/register	User registration with validation	Public
POST	/login	User login with JWT token generation	Public
POST	/logout	User logout with token clearing	Public
GET	/me	Get current user profile	Protected
PUT	/profile	Update user profile	Protected

Table 23: Authentication API Endpoints

API Endpoints

Authentication Flow

1. User submits credentials through registration or login form
2. Server validates input data and checks business rules
3. For login: verify password hash and generate JWT token
4. For registration: hash password, create user, and generate token
5. Set HTTP-only cookie with JWT token for secure session management
6. Return user data (excluding sensitive information like password)
7. Implement proper error handling for invalid credentials or duplicate users

8.3.2 Main API Routes

File: `api.js`

Purpose: Core application API endpoints for services, bookings, and payments.

Method	Endpoint	Purpose	Protection
GET	/services	Fetch all services with filtering and pagination	Public
GET	/services/:id	Get specific service details	Public
POST	/services	Create new service	Protected
PUT	/services/:id	Update service	Protected (owner only)
DELETE	/services/:id	Delete service	Protected (owner only)

Table 24: Service Management API Endpoints

Service Management Endpoints

Method	Endpoint	Purpose	Protection
POST	/bookings	Create new booking	Protected
GET	/bookings	Get user's bookings	Protected
GET	/bookings/:id	Get specific booking	Protected
PUT	/bookings/:id/confirm	Confirm booking	Protected
PUT	/bookings/:id/cancel	Cancel booking	Protected

Table 25: Booking Management API Endpoints

Booking Management Endpoints

Method	Endpoint	Purpose	Protection
POST	/payments	Process payment	Protected
GET	/payments/:paymentId	Get payment status	Protected
GET	/my-payments	Get user's payment history	Protected

Table 26: Payment Processing API Endpoints

Payment Processing Endpoints

Method	Endpoint	Purpose	Protection
POST	/wishlist/add	Add service to wishlist	Protected
DELETE	/wishlist/remove/:serviceId	Remove from wishlist	Protected
GET	/wishlist	Get user's wishlist	Protected
GET	/wishlist/check/:serviceId	Check if service is in wishlist	Protected

Table 27: Wishlist Management API Endpoints

Wishlist Management Endpoints

8.4 Middleware

Directory: `server/middleware/`

Purpose: Implements cross-cutting concerns like authentication, validation, and error handling.

8.4.1 Authentication Middleware

File: `auth.js`

Purpose: Verifies JWT tokens and protects routes from unauthorized access.

Key Features

- **Token Verification:** Validates JWT tokens from HTTP-only cookies
- **User Context:** Attaches authenticated user information to request object
- **Route Protection:** Prevents unauthorized access to protected endpoints
- **Error Handling:** Provides clear error messages for authentication failures

Authentication Functions

- **verifyToken():** Extracts and verifies JWT from cookies or Authorization headers
- **checkAuth():** Ensures user exists and attaches user object to request
- **requireAuth():** Middleware wrapper for protected routes

Authentication Process

1. Extract JWT token from HTTP-only cookies or Authorization header
2. Verify token signature using JWT secret key
3. Decode token to extract user ID and other claims
4. Query database to find user by decoded token ID
5. Attach complete user object to request for downstream middleware
6. Call next middleware in chain or return authentication error

8.4.2 File Upload Middleware

File: `upload.js`

Purpose: Handles file uploads using Multer middleware for property images.

Setting	Value
Storage Type	Local disk storage
Upload Directory	uploads/
File Naming	Timestamp + original name
File Types	Images only (jpg, jpeg, png, gif)
Size Limit	5MB per file
Field Name	'images' (multiple files)

Table 28: File Upload Configuration

Upload Configuration

9 Frontend Architecture

Technology Stack: React 18, React Router, Context API, CSS Modules

Architecture Pattern: Component-based architecture with centralized state management

9.1 Application Entry Point

File: `Client/src/main.jsx`
Purpose: React application initialization and root component rendering.

Core Functionality

- **React Initialization:** Imports React 18 and ReactDOM for modern rendering
- **Root Rendering:** Renders the main App component using createRoot API
- **DOM Mounting:** Mounts application to DOM element with id 'root'
- **Strict Mode:** Enables React.StrictMode for development checks

9.2 Main App Component

File: `Client/src/App.jsx`
Purpose: Root component that orchestrates routing and global providers.

Application Structure

- **Context Providers:** Wraps application in AuthProvider for global authentication state
- **Routing Setup:** Configures React Router with BrowserRouter for client-side navigation
- **Route Definitions:** Defines all application routes and their corresponding components
- **Global Styles:** Imports and applies global CSS styles and theme configurations
- **Error Boundaries:** Implements error handling for robust user experience

9.3 Context Management

Directory: `Client/src/context/`
Purpose: Centralized state management using React Context API.

9.3.1 Authentication Context

File: `AuthContext.js`
Purpose: Global state management for user authentication and session handling.

Property	Type	Description
user	Object/null	Current authenticated user object
loading	Boolean	Authentication check status
isAuthenticated	Boolean	User authentication state
error	String/null	Authentication error messages

Table 29: Authentication Context State

State Properties

Method	Parameters	Purpose
login()	credentials	Handles user login process
logout()	none	Handles user logout and cleanup
checkAuth()	none	Verifies current authentication status
register()	userData	Handles new user registration
updateProfile()	profileData	Updates user profile information

Table 30: Authentication Context Methods

Authentication Methods

Key Features

- **Persistent Sessions:** Maintains authentication state across page refreshes
- **Global Access:** Provides authentication methods to all components via useAuth hook
- **Loading States:** Manages loading indicators during authentication operations
- **Token Management:** Handles JWT token storage and retrieval securely
- **Error Handling:** Comprehensive error management for authentication failures

9.3.2 4.4 Page Components (Client/src/pages/)

4.4.1 Home Page (Home/Home.jsx) **Purpose:** Landing page with hero section and service overview.

Components Used: - Header: Navigation and user menu - HeroSection: Main banner with search functionality - ServiceCards: Available service categories - HowItWorks: Process explanation - TrustBadges: Trust indicators - Footer: Site footer with links

4.4.2 Authentication Pages Login Page (Login/Login.jsx): - User login form with email and password - Form validation and error handling - Redirects to dashboard on successful login - Links to registration page

Register Page (Register/Register.jsx): - User registration form with multiple fields - Password confirmation validation - Terms and conditions acceptance - Automatic login after successful registration

4.4.3 Search Page (Search/Search.jsx) **Purpose:** Property search and listing display.

Key Features: - Fetches services from backend API - Implements filtering by location, price, amenities - Displays results in grid layout using ListingCard components - Handles loading states and empty results - Integrates with ServiceModal for detailed views

Data Flow: 1. Fetch services from /api/services endpoint 2. Transform backend data to frontend format 3. Apply client-side filters 4. Render filtered results 5. Handle user interactions (view details, book, save)

4.4.4 Payment Page (Payment/Payment.jsx) **Purpose:** Multi-step payment process for property bookings.

Step Structure: 1. Personal Information: User details collection 2. Payment Method: Payment option selection 3. Confirmation: Review and submit

Payment Flow: 1. Validate service ID and user authentication 2. Create booking via POST /api/bookings 3. Process payment via POST /api/payments 4. Handle success/error responses 5. Navigate to dashboard on completion

Components: - PaymentStepIndicator: Progress visualization - PersonalInfoStep: Personal details form - PaymentMethodStep: Payment method selection - ConfirmationStep: Final review - PaymentSidebar: Booking summary - FormNavigation: Step navigation controls

4.4.5 Dashboard Page (Dashboard/Dashboard.jsx) **Purpose:** User dashboard with multiple views and functionalities.

Dashboard Views: - Overview: Summary statistics and recent activity - MyListings: User's property listings management - PaymentHistory: Transaction history and details - Wishlist: Saved properties - Profile: User profile management

State Management: - activeView: Current dashboard section - userData: User profile information - listings: User's property listings - payments: Payment transaction history - wishlist: Saved properties list

4.4.6 Service Provision Page (ProvideService/ProvideService.jsx) **Purpose:** Multi-step form for property listing creation.

Form Steps: 1. Basic Information: Title, type, description 2. Location Details: Address and area information 3. Pricing & Availability: Rent and date ranges 4. Property Details: Bedrooms, bathrooms, features 5. Amenities & Features: Available facilities 6. Photos & Contact: Images and contact information 7. Review & Submit: Final review before submission

Form Management: - Comprehensive form validation - Step-by-step navigation - Data persistence across steps - Image upload handling - Error handling and user feedback

9.3.3 4.5 Reusable Components (Client/src/components/)

4.5.1 Header Component (Header/Header.jsx) **Purpose:** Main navigation bar with authentication-aware menu.

Features: - Responsive navigation menu - User authentication status display - Drop-down menu for authenticated users - Mobile-friendly hamburger menu - Logo and brand navigation

4.5.2 ListingCard Component (ListingCard/ListingCard.jsx) **Purpose:** Property listing display card with interactive features.

Features: - Property image display with fallback - Property details (title, location, price, availability) - Action buttons (View Details, Book, Save) - Wishlist integration - Owner listing detection - Responsive design

Interaction Flow: - View Details: Opens ServiceModal or navigates to payment - Book: Navigates to payment page with property data - Save: Adds/removes from wishlist via API

4.5.3 ServiceModal Component (ServiceModal/ServiceModal.jsx) Purpose: Detailed property information modal dialog.

Content Sections: - Image gallery with main image and thumbnails - Property details (type, location, features) - Availability and pricing information - Amenities grid with icons - Property description - Contact information - Action buttons

4.5.4 FilterSidebar Component (FilterSidebar/FilterSidebar.jsx) Purpose: Search filtering interface for property listings.

Filter Options: - Location filters (district, area) - Price range (minimum and maximum) - Date range (availability period) - Property features (verified hosts, hygiene badge) - Real-time filter application

9.3.4 4.6 Utility Functions (Client/src/utils/)

4.6.1 Validation Utilities (validation.js) Purpose: Form validation functions for payment and other forms.

Validation Functions: - validateStep(): Step-specific validation for payment form - Email validation with regex patterns - Phone number validation for Bangladesh format - Required field validation - Payment method specific validation

=====

9.4 5. DATABASE IMPLEMENTATION

9.4.1 5.1 MongoDB Connection

Location: server/server.js **Connection String:** mongodb://localhost:27017/nestup_bd
ODM: Mongoose for schema definition and data modeling

Connection Process: 1. Load MongoDB URI from environment variables 2. Use mongoose.connect() with connection options 3. Handle connection events (connected, error, disconnected) 4. Implement graceful shutdown on process termination

9.4.2 5.2 Database Schema Design

5.2.1 Collections Overview

- **users:** User accounts and profiles
- **services:** Property listings and details
- **bookings:** Booking requests and status
- **payments:** Payment transactions and history

5.2.2 Relationships

- User → Services (One-to-Many): User can own multiple properties
- Service → Bookings (One-to-Many): Property can have multiple bookings
- User → Bookings (One-to-Many): User can make multiple bookings
- Booking → Payment (One-to-One): Each booking has one payment
- User → Payments (One-to-Many): User can have multiple payments

5.2.3 Indexes

- Users: email (unique), createdAt
- Services: location.district, location.area, price, createdAt
- Bookings: user, service, startDate, endDate
- Payments: user, booking, status, createdAt

9.4.3 5.3 Data Validation

Schema-Level Validation: - Required fields enforcement - Data type validation - Enum value restrictions - String length limits - Number range validation - Email format validation

Application-Level Validation: - Business logic validation - Cross-field validation - Custom validation functions - Error message customization

=====

9.5 6. API ENDPOINTS AND DATA FLOW

9.5.1 6.1 Authentication Flow

Registration Process: 1. Frontend: User submits registration form 2. Frontend: POST /api/auth/register with user data 3. Backend: Validate input data 4. Backend: Hash password using bcrypt 5. Backend: Create user in database 6. Backend: Generate JWT token 7. Backend: Set HTTP-only cookie with token 8. Backend: Return user data (excluding password) 9. Frontend: Update AuthContext with user data 10. Frontend: Redirect to dashboard

Login Process: 1. Frontend: User submits login form 2. Frontend: POST /api/auth/login with credentials 3. Backend: Find user by email 4. Backend: Compare password with stored hash 5. Backend: Generate JWT token if valid 6. Backend: Set HTTP-only cookie with token 7. Backend: Return user data 8. Frontend: Update AuthContext 9. Frontend: Redirect to intended page

9.5.2 6.2 Service Management Flow

Service Creation: 1. Frontend: User fills ProvideService form 2. Frontend: Upload images via FormData 3. Frontend: POST /api/services with service data 4. Backend: Validate user authentication 5. Backend: Process uploaded images 6. Backend: Create service in database 7. Backend: Return created service data 8. Frontend: Show success message and redirect

Service Search: 1. Frontend: User applies filters on Search page 2. Frontend: GET /api/services with query parameters 3. Backend: Build MongoDB filter object 4. Backend: Execute paginated query with population 5. Backend: Return services array with pagination info 6. Frontend: Transform data for display 7. Frontend: Render ListingCard components

9.5.3 6.3 Booking and Payment Flow

Complete Booking Process: 1. Frontend: User clicks “Book” on ListingCard 2. Frontend: Navigate to Payment page with property data 3. Frontend: User completes payment form steps 4. Frontend: POST /api/bookings to create booking 5. Backend: Validate service exists and availability 6. Backend: Check for booking conflicts 7. Backend: Calculate total amount and fees 8. Backend: Create booking with ‘pending’ status 9. Backend: Return booking data with ID 10. Frontend: POST /api/payments with booking ID 11. Backend: Validate payment details 12. Backend: Create payment record 13. Backend: Update booking payment status 14. Backend: Return payment confirmation 15. Frontend: Show success message 16. Frontend: Navigate to dashboard

9.5.4 6.4 Wishlist Management

Add to Wishlist: 1. Frontend: User clicks “Save” on ListingCard 2. Frontend: POST /api/wishlist/add with serviceId 3. Backend: Validate authentication and service 4. Backend: Check if already in wishlist 5. Backend: Add to user’s wishlist array 6. Backend: Return success response 7. Frontend: Update UI state

9.6 7. AUTHENTICATION SYSTEM

9.6.1 7.1 JWT Implementation

Token Generation: - Payload: { userId, email, iat, exp } - Secret: Stored in environment variables - Expiration: 7 days - Storage: HTTP-only cookies for security

Token Verification: - Middleware: verifyToken() function - Extraction: From cookies or Authorization header - Validation: JWT signature and expiration - User Lookup: Find user by decoded ID - Request Attachment: Add user object to req.user

9.6.2 7.2 Password Security

Hashing: - Algorithm: bcrypt with salt rounds (10) - Timing: Pre-save middleware in User model - Comparison: comparePassword() method - Storage: Never store plain text passwords

9.6.3 7.3 Route Protection

Protected Routes: - All routes requiring authentication use verifyToken middleware - Additional checkAuth middleware ensures user exists - Role-based access control for admin functions - Owner-only access for service management

9.7 8. PAYMENT SYSTEM

9.7.1 8.1 Payment Methods

Supported Methods: - Mobile Banking (bKash, Nagad, Rocket) - Bank Transfer - Credit/Debit Card - Cash Payment

Method-Specific Data: - Mobile Banking: Phone number, transaction ID - Bank Transfer: Bank name, account number - Credit Card: Card number, expiry, CVV, holder name - Cash: Contact information for arrangement

9.7.2 8.2 Payment Processing

Payment States: - pending: Initial state after creation - processing: Payment being processed - completed: Payment successful - failed: Payment failed with reason - refunded: Payment refunded - cancelled: Payment cancelled by user

Processing Flow: 1. Create booking first 2. Validate payment details 3. Create payment record 4. Process payment (simulated) 5. Update payment status 6. Update booking payment status 7. Send confirmation

9.7.3 8.3 Payment History

Data Tracking: - All payment transactions stored - Status change history - Payment method details - Associated booking information - Refund tracking

Dashboard Integration: - Payment history view in dashboard - Filtering by status and date - Detailed payment information - Download/export capabilities

=====

9.8 9. FILE-BY-FILE ANALYSIS

9.8.1 9.1 Backend Files

server/server.js **Purpose:** Main server application entry point **Key Functions:** - Express app initialization - Middleware configuration - Database connection - Route mounting - Server startup **Dependencies:** express, mongoose, cors, cookie-parser, dotenv **Exports:** None (entry point)

server/models/User.js **Purpose:** User data model and schema **Key Functions:** - User schema definition - Password hashing middleware - Password comparison method - Profile data structure **Dependencies:** mongoose, bcryptjs **Exports:** User model

server/models/Service.js **Purpose:** Property service data model **Key Functions:** - Service schema definition - Location and pricing structure - Amenities and features - Review system integration **Dependencies:** mongoose **Exports:** Service model

server/models/Booking.js **Purpose:** Booking management data model **Key Functions:** - Booking schema definition - Date validation - Status management - Availability checking **Dependencies:** mongoose **Exports:** Booking model

server/models/Payment.js **Purpose:** Payment processing data model **Key Functions:** - Payment schema definition - Payment method handling - Status tracking - Refund management **Dependencies:** mongoose **Exports:** Payment model

server/routes/auth.js **Purpose:** Authentication route handlers **Key Functions:** - User registration - User login/logout - Profile management - JWT token handling **Dependencies:** express, bcryptjs, jsonwebtoken, User model **Exports:** Express router

server/routes/api.js **Purpose:** Main API route handlers **Key Functions:** - Service CRUD operations - Booking management - Payment processing - Wishlist operations **Dependencies:** express, all models, auth middleware **Exports:** Express router

server/middleware/auth.js **Purpose:** Authentication middleware **Key Functions:** - JWT token verification - User authentication checking - Request user attachment **Dependencies:** jsonwebtoken, User model **Exports:** verifyToken, checkAuth functions

server/middleware/upload.js **Purpose:** File upload middleware **Key Functions:** - Multer configuration - File storage setup - File type filtering - Size limitations **Dependencies:** multer, path **Exports:** upload middleware

9.8.2 9.2 Frontend Files

Client/src/main.jsx **Purpose:** React application entry point **Key Functions:** - React DOM rendering - App component mounting **Dependencies:** react, react-dom, App component **Exports:** None (entry point)

Client/src/App.jsx **Purpose:** Root application component **Key Functions:** - Router setup - Global providers - Route definitions **Dependencies:** react-router-dom, AuthProvider, all page components **Exports:** App component

Client/src/context/AuthContext.js **Purpose:** Authentication context provider **Key Functions:** - Global auth state management - Login/logout functions - Authentication checking **Dependencies:** react, createContext, useState, useEffect **Exports:** AuthProvider, useAuth hook

Client/src/pages/Home/Home.jsx **Purpose:** Landing page component **Key Functions:** - Hero section display - Service overview - Navigation integration **Dependencies:** react, Header, HeroSection, ServiceCards, etc. **Exports:** Home component

Client/src/pages/Login/Login.jsx **Purpose:** User login page **Key Functions:** - Login form handling - Authentication API calls - Error handling - Redirect logic **Dependencies:** react, react-router-dom, AuthContext **Exports:** Login component

Client/src/pages/Register/Register.jsx **Purpose:** User registration page **Key Functions:** - Registration form handling - Form validation - API integration - Success handling **Dependencies:** react, react-router-dom, AuthContext **Exports:** Register component

Client/src/pages/Search/Search.jsx **Purpose:** Property search and listing page **Key Functions:** - Service data fetching - Filter application - Results display - Modal integration **Dependencies:** react, AuthContext, FilterSidebar, ListingCard, ServiceModal **Exports:** Search component

Client/src/pages/Payment/Payment.jsx **Purpose:** Multi-step payment processing page **Key Functions:** - Payment form management - Step navigation - API integration - Booking creation **Dependencies:** react, react-router-dom, AuthContext, payment components **Exports:** Payment component

Client/src/pages/Dashboard/Dashboard.jsx **Purpose:** User dashboard with multiple views **Key Functions:** - View management - Data fetching - User profile display - Navigation handling **Dependencies:** react, AuthContext, dashboard view components **Exports:** Dashboard component

Client/src/pages/ProvideService/ProvideService.jsx **Purpose:** Property listing creation page **Key Functions:** - Multi-step form handling - Image upload - Form validation - Service creation API **Dependencies:** react, react-router-dom, AuthContext **Exports:** ProvideService component

Client/src/components/Header/Header.jsx **Purpose:** Main navigation header **Key Functions:** - Navigation menu - User authentication display - Mobile responsiveness **Dependencies:** react, react-router-dom, AuthContext **Exports:** Header component

Client/src/components/ListingCard/ListingCard.jsx **Purpose:** Property listing display card **Key Functions:** - Property information display - Action button handling - Wishlist integration - Navigation to payment **Dependencies:** react, react-router-dom **Exports:** ListingCard component

Client/src/components/ServiceModal/ServiceModal.jsx **Purpose:** Detailed property information modal **Key Functions:** - Detailed property display - Image gallery - Contact information - Modal controls **Dependencies:** react **Exports:** ServiceModal component

Client/src/components/FilterSidebar/FilterSidebar.jsx **Purpose:** Search filter interface **Key Functions:** - Filter option display - Filter state management - Real-time filtering **Dependencies:** react **Exports:** FilterSidebar component

=====

9.9 10. FRONTEND-BACKEND CONNECTION FLOW

9.9.1 10.1 API Communication Pattern

Standard Request Flow: 1. Frontend component initiates action (user interaction) 2. Component calls `fetch()` with appropriate endpoint 3. Request includes authentication cookies automatically 4. Backend middleware verifies authentication 5. Route handler processes request 6. Database operations performed if needed 7. Response sent back to frontend 8. Frontend updates UI based on response

9.9.2 10.2 Authentication Integration

Cookie-Based Authentication: - JWT tokens stored in HTTP-only cookies - Automatic inclusion in all requests - No manual token management in frontend - Secure against XSS attacks

AuthContext Integration: - Global authentication state - Automatic login status checking - Protected route handling - User data availability across components

9.9.3 10.3 Error Handling

Frontend Error Handling: - Try-catch blocks around API calls - User-friendly error messages - Loading state management - Fallback UI for errors

Backend Error Handling: - Consistent error response format - HTTP status codes - Detailed error messages - Logging for debugging

9.9.4 10.4 Data Transformation

Backend to Frontend: - MongoDB ObjectId to string conversion - Date formatting - Price formatting - Image URL construction

Frontend to Backend: - Form data serialization - Date object conversion - File upload handling - Validation before sending

9.10 11. CORE FUNCTIONALITIES

9.10.1 11.1 User Management

Registration and Authentication: - Secure user registration with validation - JWT-based authentication system - Password hashing and security - Profile management capabilities

User Roles: - Regular users: Can search, book, and manage bookings - Property owners: Can list properties and manage listings - Admins: Can manage all aspects of the platform

9.10.2 11.2 Property Management

Property Listing: - Multi-step property creation form - Image upload and management - Detailed property information - Availability calendar

Property Search: - Advanced filtering options - Location-based search - Price range filtering - Amenity-based filtering

9.10.3 11.3 Booking System

Booking Process: - Date availability checking - Conflict prevention - Booking confirmation system - Status tracking

Booking Management: - User booking history - Booking status updates - Cancellation handling - Owner booking management

9.10.4 11.4 Payment Processing

Payment Methods: - Multiple payment options - Secure payment processing - Payment status tracking - Refund handling

Financial Management: - Payment history tracking - Transaction records - Revenue analytics - Fee calculation

9.10.5 11.5 Wishlist System

Wishlist Management: - Save favorite properties - Easy access to saved items - Wishlist sharing capabilities - Notification for saved items

9.11 12. MONGODB COLLECTIONS AND SCHEMA

9.11.1 12.1 Collection Initialization

Location: Models are defined in server/models/ directory **Initialization:** Mongoose automatically creates collections when first document is inserted **Connection:** Established in server/server.js using mongoose.connect()

9.11.2 12.2 Collection Details

12.2.1 users Collection Schema File: server/models/User.js **Purpose:** Store user account information and profiles **Key Fields:** - `__id`: MongoDB ObjectId (auto-generated) - `name`: User's full name - `email`: Unique email address - `password`: Hashed password - `role`: User role (user/admin) - `profile`: Nested object with personal information - `createdAt`, `updatedAt`: Timestamps

Indexes: - `email`: Unique index for fast lookup - `createdAt`: For sorting and analytics

12.2.2 services Collection Schema File: server/models/Service.js **Purpose:** Store property listing information **Key Fields:** - `__id`: MongoDB ObjectId (auto-generated) - `title`: Property title - `propertyType`: Type of property - `location`: Nested object with address details - `price`: Monthly rent amount - `availability`: Date range and status - `propertyDetails`: Bedrooms, bathrooms, etc. - `amenities`: Available facilities - `owner`: Reference to users collection - `createdAt`, `updatedAt`: Timestamps

Indexes: - `location.district`: For location-based queries - `location.area`: For area-specific searches - `price`: For price range filtering - `owner`: For owner's listings

12.2.3 bookings Collection **Schema File:** server/models/Booking.js **Purpose:** Store booking requests and status **Key Fields:** - `_id`: MongoDB ObjectId (auto-generated) - `service`: Reference to services collection - `user`: Reference to users collection - `startDate`, `endDate`: Booking period - `totalAmount`: Total booking cost - `status`: Booking status - `paymentStatus`: Payment status - `confirmationCode`: Unique booking code - `createdAt`, `updatedAt`: Timestamps

Indexes: - `user`: For user's booking history - `service`: For property booking history - `startDate`, `endDate`: For availability checking

12.2.4 payments Collection **Schema File:** server/models/Payment.js **Purpose:** Store payment transaction records **Key Fields:** - `_id`: MongoDB ObjectId (auto-generated) - `booking`: Reference to bookings collection - `user`: Reference to users collection - `service`: Reference to services collection - `amount`: Payment amount - `paymentMethod`: Method used for payment - `status`: Payment status - `paymentDetails`: Method-specific details - `createdAt`, `updatedAt`: Timestamps

Indexes: - `user`: For user's payment history - `booking`: For booking-payment relationship - `status`: For status-based queries - `createdAt`: For chronological sorting

9.11.3 12.3 Data Relationships

Reference Relationships: - User → Services (One-to-Many) - User → Bookings (One-to-Many) - User → Payments (One-to-Many) - Service → Bookings (One-to-Many) - Booking → Payment (One-to-One)

Population Usage: - Services populated with owner information - Bookings populated with user and service details - Payments populated with booking and user information

9.11.4 12.4 Database Operations

Common Operations: - CRUD operations for all collections - Complex queries with filtering and sorting - Aggregation pipelines for analytics - Population for related data - Indexing for performance optimization

Performance Considerations: - Proper indexing on frequently queried fields - Pagination for large result sets - Efficient query patterns - Connection pooling - Query optimization

9.12 CONCLUSION

This comprehensive analysis covers the complete architecture and implementation of the NestUp BD platform. The system demonstrates a well-structured full-stack application with proper separation of concerns, secure authentication, robust data modeling, and user-friendly interfaces. The modular design allows for easy maintenance and future enhancements while providing a solid foundation for a property rental platform.

10 Technical Analysis Summary

10.1 Architecture Strengths

Backend Architecture

- **Modular Design:** Clear separation of concerns with organized route handlers, middleware, and models
- **RESTful API:** Consistent API design following REST principles for predictable endpoints
- **Middleware Stack:** Comprehensive middleware for authentication, file uploads, and request processing
- **Database Integration:** Efficient MongoDB integration with Mongoose ODM for schema validation
- **Security Implementation:** Robust security measures including JWT authentication and input validation

Frontend Architecture

- **Modern React Patterns:** Utilizes React 18 features with hooks and context for state management
- **Component Reusability:** Well-structured component hierarchy with reusable UI elements
- **Routing System:** Comprehensive routing with protected routes and nested navigation
- **User Experience:** Smooth animations and transitions using Framer Motion
- **Responsive Design:** Mobile-first approach with responsive layouts

10.2 Business Logic Implementation

Core Workflows

- **Service Management:** Complete CRUD operations for property listings with image support
- **Booking System:** Comprehensive booking workflow with validation and approval processes
- **Payment Integration:** Multiple payment method support including mobile banking
- **Review System:** Dual review system for both guests and hosts
- **User Management:** Secure authentication with profile management

Data Flow

- **API Communication:** Consistent data exchange between frontend and backend
- **State Synchronization:** Real-time updates across different components
- **Error Handling:** Comprehensive error management at all levels
- **Validation Pipeline:** Multi-layer validation from client to database

10.3 Security and Performance

Security Measures

- **Authentication Security:** JWT tokens with HTTP-only cookies and secure flags
- **Input Validation:** Comprehensive validation on both client and server sides
- **File Upload Security:** Restricted file types and size limits with secure storage
- **Database Security:** Secure connection handling and query sanitization
- **Access Control:** Role-based access with ownership validation

Performance Optimizations

- **Database Indexing:** Optimized queries with appropriate indexes
- **Connection Pooling:** Efficient database connection management
- **Asset Optimization:** Vite build system with code splitting and optimization
- **Lazy Loading:** Component-level lazy loading for better performance
- **Caching Strategy:** Efficient caching of static assets and API responses

11 Recommendations and Future Enhancements

11.1 Technical Improvements

Backend Enhancements

- **API Documentation:** Implement Swagger/OpenAPI documentation
- **Rate Limiting:** Add rate limiting middleware for API protection
- **Logging System:** Implement comprehensive logging with Winston or similar
- **Testing Suite:** Add unit and integration tests for critical functionality
- **Caching Layer:** Implement Redis for session and data caching

Frontend Enhancements

- **State Management:** Consider Redux Toolkit for complex state management
- **Performance Monitoring:** Implement performance monitoring and analytics
- **Progressive Web App:** Add PWA features for mobile experience
- **Accessibility:** Enhance accessibility compliance (WCAG guidelines)
- **Internationalization:** Add multi-language support

11.2 Business Feature Enhancements

Platform Features

- **Real-time Messaging:** Implement chat system between hosts and guests
- **Advanced Search:** Add more sophisticated filtering and search capabilities
- **Recommendation Engine:** Implement AI-based service recommendations
- **Mobile Application:** Develop native mobile applications
- **Analytics Dashboard:** Add comprehensive analytics for hosts

Integration Opportunities

- **Payment Gateways:** Integrate additional payment providers
- **Map Integration:** Add interactive maps for location visualization
- **Social Media:** Implement social media integration and sharing
- **Email Notifications:** Add comprehensive email notification system
- **SMS Integration:** Implement SMS notifications for important updates

12 Conclusion

The NestUp BD platform demonstrates a well-architected full-stack application with modern development practices and comprehensive functionality. The codebase exhibits strong separation of concerns, security-first approach, and scalable architecture suitable for a property booking platform.

The codebase follows modern web development best practices with React for the frontend, Node.js/Express for the backend, and MongoDB for data persistence. The authentication system ensures security, while the payment system provides multiple options for users. The comprehensive API design enables smooth communication between frontend and backend components.

Key Achievements

- **Complete Functionality:** Full-featured booking platform with all essential components
- **Security Implementation:** Robust security measures protecting user data and transactions
- **Modern Technology Stack:** Utilizes current best practices and technologies
- **User Experience:** Intuitive interface with smooth interactions and responsive design
- **Scalable Architecture:** Design supports future growth and feature additions

Technical Excellence The application successfully implements complex business logic while maintaining code quality, security standards, and performance optimization. The modular architecture facilitates maintenance and future enhancements, making it a solid foundation for a commercial property booking platform.

This documentation serves as a complete reference for understanding, maintaining, and extending the NestUp BD platform.